DBMS Laboratory · Mini project

Reimbursement Management System

Project report: problem, requirements, ER design, relational schema, functional dependencies and normalization

MIS number	Name
612415001	Abdurrahman Shaikh
612415057	Dhruv Agarwal
612415044	Mrudula Dagale
612415093	Makarand S Kulkarni

October 2026

Contents

1. Problem statement....3

2. Objectives....3

3. Functional requirements....3

3.1 Users and roles....3

3.2 Requirements by role....4

3.3 Data rules the database must enforce....4

3.4 Non-functional requirements....5

4. ER diagram....6

4.1 Notation used....8

4.2 Entity sets....8

4.3 Relationship sets....9

4.4 Specializations and aggregation....10

4.5 Changes from our ER diagram assignment....11

5. Relational schemas....12

5.1 Reduction rules applied....12

5.2 Resulting relation schemas....12

6. Functional dependencies....15

7. Normalization up to 3NF....16

7.1 Definitions....16

7.2 Every relation of the final schema....16

7.3 Walkthrough from an unnormalized claim form....16

8. Implementation....18

8.1 Architecture....18

8.2 Where each rule is enforced....18

8.3 Screens....19

8.4 Running the project....21

9. Conclusion....22

1. Problem statement

Organisations repay employees for money they spend on the organisation's behalf: travel to clients, medical treatment covered by policy, and business expenses such as client meetings. In many offices this still runs on paper forms, e-mailed spreadsheets and bills stapled together. That process has well-known failures:

- Receipts and forms get lost, and nobody can say where a claim is stuck.
- Approval authority is informal: a junior manager can approve a large claim, or a manager can approve their own.
- The same claim can be paid twice, and payments cannot be traced to a bank transfer.
- Totals are retyped by hand, so the approved amount and the paid amount can disagree with the bills.
- Finance cannot answer simple questions (spend by department, by month, by expense type) without re-entering data.

The problem is to design and implement a database application in which employees submit claims made of itemised expenses, managers approve or reject them within their authority, and the finance team pays approved claims to the employee's bank account. The database must be the single source of truth and must itself refuse data that breaks the organisation's rules, whichever program writes to it.

2. Objectives

- Model the reimbursement process with an E-R diagram using the notation of the course textbook (Silberschatz, Korth & Sudarshan, 7th ed.).
- Reduce the E-R diagram to relation schemas with the textbook's rules, identify every functional dependency, and normalize every relation to at least 3NF (all relations reach BCNF).
- Enforce the business rules inside MySQL with keys, foreign keys, CHECK constraints and triggers, so that no client can bypass them.
- Build a role-based web application (React front end, Node.js back end) for applicants, managers and admins.
- Keep a complete trail of who submitted, reviewed and paid each claim, and when.
- Provide reports on spending by department, claim type, expense category and month.

3. Functional requirements

3.1 Users and roles

Every user is an employee. An employee can hold any combination of three roles, which matches the overlapping specialization in the ER diagram:

Role	Who	What they do
Applicant	Employees with a registered payout bank account	File claims and track them
Manager	Employees with an approval limit	Approve or reject other employees' claims up to their limit
Admin	Finance / administration staff	Pay approved claims; maintain employees, roles and departments

3.2 Requirements by role

ID	Requirement
FR1	All users sign in with their e-mail address and password; the menu shows only the functions of their roles.
FR2	An applicant submits a claim of exactly one type (travel, medical or business) with the details of that type and one or more expense items (category, description, date, amount, optional receipt file).
FR3	An applicant sees all their claims with type, date, number of items, total, status and progress (submitted, reviewed by whom and when, paid when and with which bank reference).
FR4	An applicant can withdraw a claim until a manager has reviewed it.
FR5	A manager sees a queue of submitted claims that they are allowed to decide: not their own, and with a total within their approval limit. Claims from their direct reports are listed first.
FR6	A manager approves or rejects a claim; a rejection must state the reason. Each claim is decided once.
FR7	A manager sees the claims they have reviewed.
FR8	An admin sees approved claims waiting for payment, with the payee's bank account and IFSC code, and records each payment with its bank transfer reference (UTR).
FR9	An admin adds and edits employees: personal details, department, supervisor, any number of phone numbers, and roles (applicant with bank details, manager with approval limit, admin).
FR10	An admin adds, renames and deletes departments.
FR11	Managers and admins see reports: claims by status, approved spend by department, claim type, expense category and month, the top claimants, and the reporting hierarchy.

3.3 Data rules the database must enforce

ID	Rule
DR1	Only applicants can file claims, only managers can review them, and only admins can pay them.
DR2	Nobody reviews or pays their own claim, and the person who approved a claim cannot also pay it.
DR3	A manager may only decide claims whose total is within their approval limit.
DR4	A claim moves only SUBMITTED → APPROVED or REJECTED, then APPROVED → PAID. Rejected and paid claims are closed.
DR5	A rejection carries remarks; an approval or rejection carries the reviewer and date; a payment carries the admin, date and a unique bank reference.
DR6	A claim has exactly one type, the details of that type only, and at least one expense item. Amounts are positive.
DR7	Once reviewed, a claim's items and details cannot change, so the approved total is the paid total.
DR8	Dates are in order (expense ≤ claim ≤ review ≤ payment) and never in the future.
DR9	E-mail addresses, bank accounts (IFSC + account number) and payment references are unique.
DR10	The reporting hierarchy has no cycles; an employee cannot supervise themselves.
DR11	Departments with employees, and employees with claim history, cannot be deleted.

3.4 Non-functional requirements

- **Integrity:** every rule above is enforced by the database, not only by the user interface.
- **Atomicity:** a claim, its details and its items are saved together or not at all.
- **Security:** passwords stored only as bcrypt hashes; parameterised SQL; the application connects with a database account that cannot change the schema.
- **Portability:** standard SQL (CHECK constraints rather than MySQL ENUMs) wherever MySQL allows it.

4. ER diagram

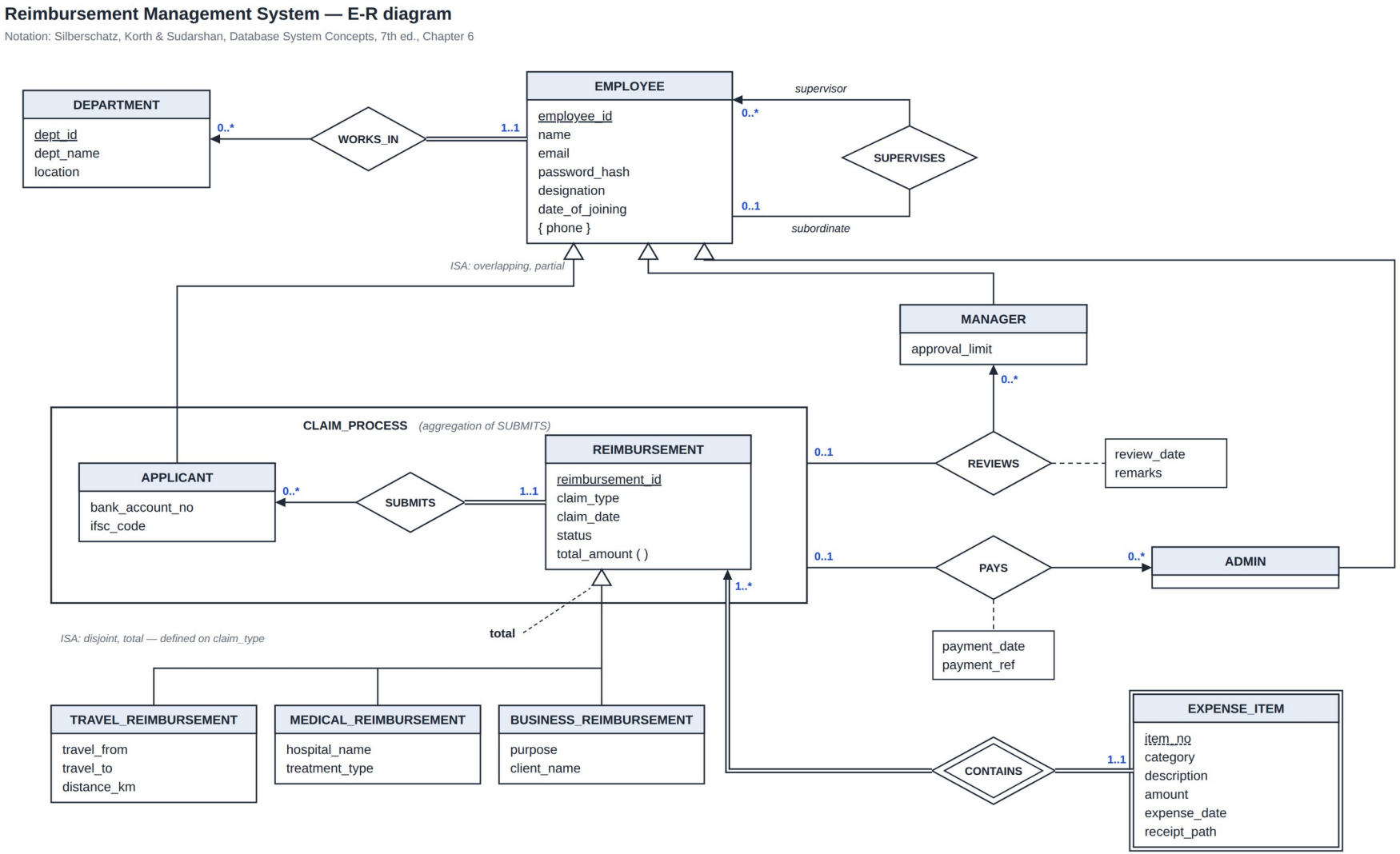


Figure 1. E-R diagram of the Reimbursement Management System (textbook notation; the symbol key is in Section 4.1). A full-resolution copy with the key drawn in is report/er/er_diagram.png.

4.1 Notation used

The diagram uses the notation of the course textbook (Chapter 6), so every symbol has the meaning given there:

Symbol in the diagram	Meaning (Silberschatz Chapter 6)
Rectangle with a shaded header	Entity set; attributes listed below the name; primary key underlined (§6.2)
Double rectangle	Weak entity set; its discriminator has a dashed underline (§6.5.3)
Diamond / double diamond	Relationship set / identifying relationship of a weak entity set
Arrow on a line (→)	The "one" side: each entity at the other end relates to at most one entity here (§6.4)
Plain line	The "many" side
Double line	Total participation: every entity of that set takes part in the relationship
l..h on a line	Minimum and maximum number of relationships each entity of that set takes part in (e.g. 1..1, 0..*)
Role label on a line (supervisor, subordinate)	Role indicator, needed when an entity set relates to itself
Rectangle linked by a dashed line to a diamond	Descriptive attributes of the relationship
{ phone }	Multivalued attribute
total_amount ( )	Derived attribute
Hollow arrowhead	Specialization (ISA), pointing to the superclass. Separate arrows = overlapping; one shared arrow = disjoint; the keyword "total" (dashed line to the arrowhead) = total specialization (§6.8)
Box around a relationship and its entity sets	Aggregation: the relationship treated as an entity set (§6.8.5)

4.2 Entity sets

Entity set	Kind	Attributes (key underlined)	Notes
DEPARTMENT	Strong	dept_id, dept_name, location	dept_name is also unique (candidate key).
EMPLOYEE	Strong; superclass	employee_id, name, email, password_hash, designation, date_of_joining, { phone }	Phone is multivalued. email is a candidate key and the login name.

Entity set	Kind	Attributes (key underlined)	Notes
APPLICANT	Subclass of EMPLOYEE	bank_account_no, ifsc_code (inherits employee_id)	An employee who may file claims; needs a payout account.
MANAGER	Subclass of EMPLOYEE	approval_limit (inherits employee_id)	May review claims up to approval_limit.
ADMIN	Subclass of EMPLOYEE	none of its own (inherits employee_id)	Exists because only admins take part in PAYS.
REIMBURSEMENT	Strong; superclass	reimbursement_id, claim_type, claim_date, status, total_ amount ( )	total_amount is derived from the items. claim_type defines the subclass.
TRAVEL_REIMBURSEMENT	Subclass of REIMBURSEMENT	travel_from, travel_to, distance_km	Details of a travel claim.
MEDICAL_REIMBURSEMENT	Subclass of REIMBURSEMENT	hospital_name, treatment_type	Details of a medical claim.
BUSINESS_REIMBURSEMENT	Subclass of REIMBURSEMENT	purpose, client_name	client_name is optional (internal business).
EXPENSE_ITEM	Weak (owner: REIMBURSEMENT)	item_no (discriminator), category, description, amount, expense_date, receipt_path	Identified by its claim plus item_no: "item 2 of claim 17".
CLAIM_PROCESS	Aggregation	APPLICANT – SUBMITS – REIMBURSEMENT treated as one unit	REVIEWS and PAYS relate to it.

4.3 Relationship sets

Each relationship is described by its mapping cardinality, the participation of each side, and how the diagram shows both. A label l..h sits on the line next to the entity set it describes: on SUBMITS, 1..1 next to REIMBURSEMENT means every claim is submitted by exactly one applicant, and 0..* next to APPLICANT means an applicant submits any number of claims.

Relationship	Connects	Mapping cardinality	Participation	How it is drawn	Becomes in the schema
WORKS_IN	EMPLOYEE – DEPARTMENT	Many-to-one	EMPLOYEE total; DEPARTMENT partial	Arrow to DEPARTMENT; double line from EMPLOYEE; 1..1 on EMPLOYEE, 0..* on DEPARTMENT	`employee.dept_id` NOT NULL, FK → department

Relationship	Connects	Mapping cardinality	Participation	How it is drawn	Becomes in the schema
SUPERVISES	EMPLOYEE (supervisor) – EMPLOYEE (subordinate)	One-to-many	Both partial	Arrow on the supervisor line; role labels; 0..* (supervisor), 0..1 (subordinate)	`employee.supervisor_id` NULL, FK → employee
SUBMITS	APPLICANT – REIMBURSEMENT	One-to-many	REIMBURSEMENT total; APPLICANT partial	Arrow to APPLICANT; double line from REIMBURSEMENT; 0..* on APPLICANT, 1..1 on REIMBURSEMENT	`reimbursement.applicant_id` NOT NULL, FK → applicant
REVIEWS	MANAGER – CLAIM_ PROCESS (aggregation)	One-to-many (one decision per claim)	Both partial	Arrow to MANAGER; plain line from the aggregation; 0..* on MANAGER, 0..1 on CLAIM_PROCESS; descriptive attributes review_date, remarks	`reimbursement.reviewed_by`, `review_ date`, `remarks` (NULL until reviewed); reviewed_by FK → manager
PAYS	ADMIN – CLAIM_PROCESS (aggregation)	One-to-many (one payment per claim)	Both partial	Arrow to ADMIN; plain line from the aggregation; 0..* on ADMIN, 0..1 on CLAIM_ PROCESS; descriptive attributes payment_ date, payment_ref	`reimbursement.paid_by`, `payment_date`, `payment_ref` (NULL until paid); paid_by FK → admin
CONTAINS (identifying)	REIMBURSEMENT – EXPENSE_ITEM (weak)	One-to-many	Both total	Double diamond; arrow and double line to REIMBURSEMENT; double line to EXPENSE_ITEM; 1..* on REIMBURSEMENT, 1..1 on EXPENSE_ITEM	No relation of its own; expense_item has PK (reimbursement_id, item_no) and FK → reimbursement ON DELETE CASCADE

4.4 Specializations and aggregation

Specialization	Subclasses	Constraints	How it is drawn	Becomes in the schema
ISA on EMPLOYEE	APPLICANT, MANAGER, ADMIN	Overlapping, partial	Three separate hollow arrows; no "total" keyword	Tables applicant, manager, admin; each PK is also an FK → employee
ISA on REIMBURSEMENT	TRAVEL_, MEDICAL_, BUSINESS_REIMBURSEMENT	Disjoint, total; attribute-defined on claim_type	One shared hollow arrow; "total" keyword joined by a dashed line	Tables travel_/medical_/business_reimbursement; each PK is also an FK → reimbursement

Aggregation. CLAIM_PROCESS treats the relationship "applicant submits reimbursement" as one unit. A manager reviews, and an admin pays, a submission: who claimed what. Modelling REVIEWS and PAYS against the aggregation (rather than as ternary relationships with APPLICANT and REIMBURSEMENT) avoids recording the applicant a second time and allowing a review whose applicant differs from the person who submitted the claim (textbook §6.8.5).

4.5 Changes from our ER diagram assignment

The diagram above refines the one submitted for the ER assignment. Every change is justified in the separate Design Decisions document (decision numbers in brackets).

Earlier diagram	Final diagram	Reason
Department was an attribute of EMPLOYEE	DEPARTMENT entity set and WORKS_IN [E1]	A department has its own attribute (location) and is shared; as an attribute it would cause a transitive dependency.
APPROVES, no attributes	REVIEWS with review_date, remarks [E8]	A manager may also reject; decisions need a date and a reason.
No way to record payment	ADMIN subclass and PAYS with payment_date, payment_ref [E5, E9]	Disbursement is a separate step by a separate person.
Bank_Account_No only	bank_account_no + ifsc_code, unique together [E6]	Account numbers are unique only within a bank.
Specialization of REIMBURSEMENT not qualified	Disjoint and total, defined on claim_type [E10]	Every claim is exactly one type.
Status values not defined	SUBMITTED, APPROVED, REJECTED, PAID with fixed transitions [E11]	Gives the workflow and its constraints.
—	password_hash on EMPLOYEE; category on EXPENSE_ITEM [E16, E14]	Sign-in; spend reports by category.
Receipt	receipt_path [E15]	Files are stored on disk; the database keeps the path.
CONTAINS: REIMBURSEMENT partial	REIMBURSEMENT total (1..*) [E13]	A claim with no items is meaningless.
Ovals and 1/N labels	Attributes inside the rectangles; arrows, double lines and l..h	Matches the course textbook's notation.

5. Relational schemas

5.1 Reduction rules applied

The relations come from the E-R diagram by the reduction rules of textbook §6.7 and §6.8.6:

ER construct	Rule (textbook)	Result
Strong entity sets DEPARTMENT, EMPLOYEE, REIMBURSEMENT	One relation with the same attributes; same primary key (§6.7.1)	department, employee, reimbursement
Multivalued attribute { phone }	New relation: owner's key + the attribute; both form the key (§6.7.2)	employee_phone (employee_id, phone)
Derived attribute total_amount ( )	Not stored; computed when needed (§6.7.2)	Function `claim_total()` and view `reimbursement_summary`
Weak entity set EXPENSE_ITEM	Owner's key + discriminator = primary key; FK to owner (§6.7.3)	expense_item (reimbursement_id, item_no, …)
Identifying relationship CONTAINS	Redundant; no relation (§6.7.5)	Represented by expense_item's foreign key
Many-to-one with total participation: WORKS_IN, SUBMITS	Merge into the "many" side (§6.7.6)	`employee.dept_id`, `reimbursement.applicant_id` (NOT NULL)
Many-to-one with partial participation: SUPERVISES, REVIEWS, PAYS	Merge into the "many" side, using NULLs (§6.7.6)	`supervisor_id`; `reviewed_by`, `review_date`, `remarks`; `paid_by`, `payment_date`, `payment_ref`
Specializations of EMPLOYEE and REIMBURSEMENT	Method 1: superclass relation plus one relation per subclass holding the superclass key (§6.8.6.1)	applicant, manager, admin; travel_/medical_/business_reimbursement
Aggregation CLAIM_PROCESS	No relation of its own; its key is the key of SUBMITS (§6.8.6.2)	Key reimbursement_id, so REVIEWS and PAYS merge into reimbursement

5.2 Resulting relation schemas

Primary keys are underlined. Foreign keys and other keys are listed under each schema. All 11 relations are implemented in db/01_schema.sql.

1. department (dept_id, dept_name, location)
UNIQUE (dept_name)

2. employee (employee_id, name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id)
UNIQUE (email); dept_id → department; supervisor_id → employee (NULL allowed)

3. employee_phone (employee_id, phone)
employee_id → employee

4. applicant (employee_id, bank_account_no, ifsc_code)
employee_id → employee; UNIQUE (ifsc_code, bank_account_no)

5. manager (employee_id, approval_limit)
employee_id → employee

6. admin (employee_id)
employee_id → employee

7. reimbursement (reimbursement_id, claim_type, applicant_id, claim_date, status, reviewed_by, review_date, remarks, paid_by, payment_date, payment_ref)

applicant_id → applicant; reviewed_by → manager (NULL allowed); paid_by → admin (NULL allowed); UNIQUE (payment_ref)

8. travel_reimbursement (reimbursement_id, travel_from, travel_to, distance_km)

reimbursement_id → reimbursement

9. medical_reimbursement (reimbursement_id, hospital_name, treatment_type)

reimbursement_id → reimbursement

10. business_reimbursement (reimbursement_id, purpose, client_name)

reimbursement_id → reimbursement

11. expense_item (reimbursement_id, item_no, category, description, amount, expense_date, receipt_path)

reimbursement_id → reimbursement

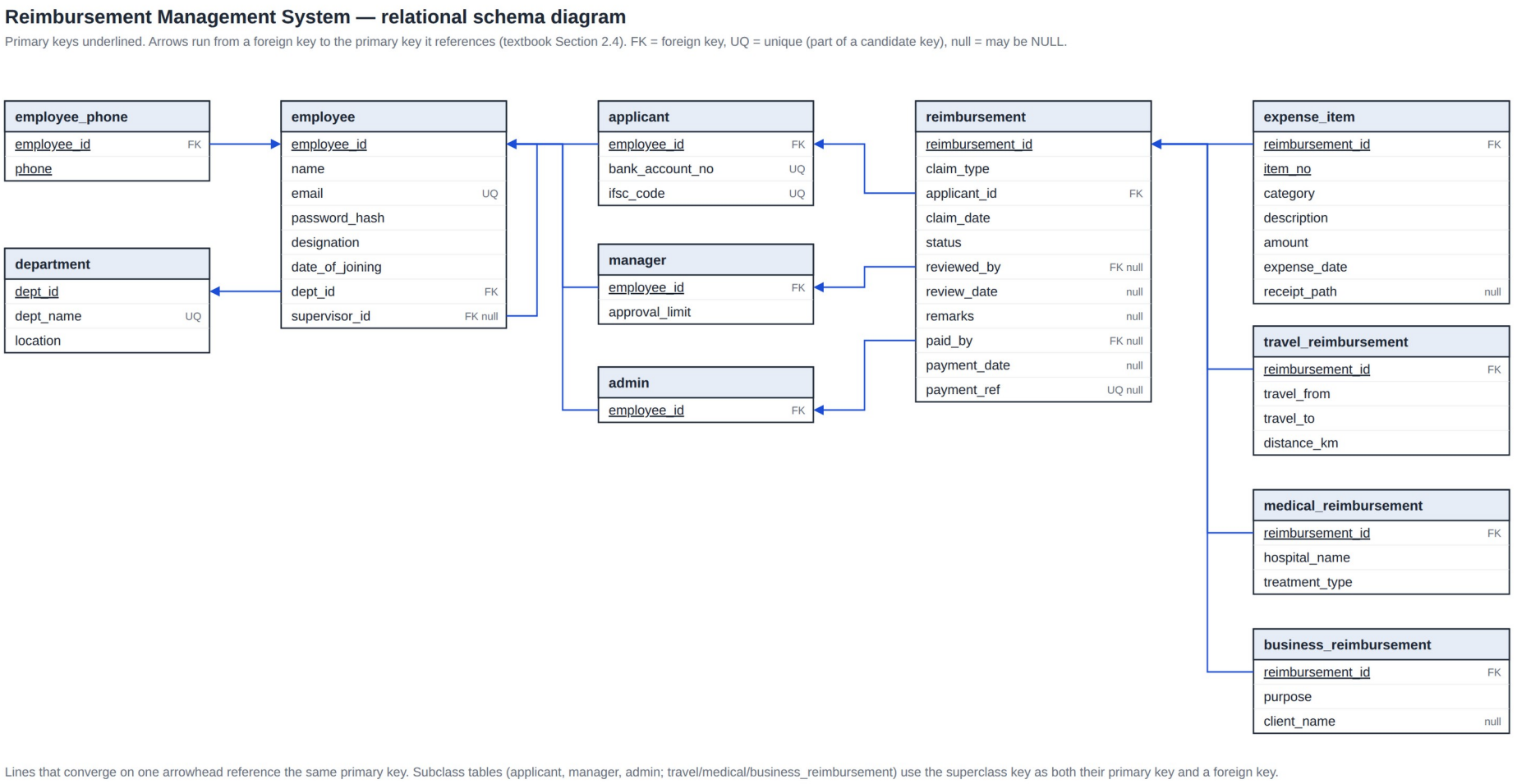


Figure 2. Schema diagram (textbook §2.4): arrows run from each foreign key to the primary key it references.

6. Functional dependencies

The functional dependencies below are the constraints that must hold on each relation; they come from the business rules, not from the sample data. Attribute closures were used to find the candidate keys (textbook §7.4.2).

Relation	Functional dependencies that must hold (F)	Candidate keys	Highest normal form
department	dept_id → dept_name, location dept_name → dept_id	{dept_id}, {dept_name}	BCNF (so also 3NF)
employee	employee_id → name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id email → employee_id	{employee_id}, {email}	BCNF (so also 3NF)
employee_phone	Only trivial FDs (both attributes form the key)	{employee_id, phone}	BCNF (so also 3NF)
applicant	employee_id → bank_account_no, ifsc_code ifsc_code, bank_account_no → employee_id	{employee_id}, {ifsc_code, bank_account_no}	BCNF (so also 3NF)
manager	employee_id → approval_limit	{employee_id}	BCNF (so also 3NF)
admin	Only trivial FDs	{employee_id}	BCNF (so also 3NF)
reimbursement	reimbursement_id → claim_type, applicant_id, claim_date, status, reviewed_by, review_date, remarks, paid_by, payment_date, payment_ref payment_ref → reimbursement_id (for paid claims; UNIQUE, NULL until paid)	{reimbursement_id}	BCNF (so also 3NF)
travel_reimbursement	reimbursement_id → travel_from, travel_to, distance_km	{reimbursement_id}	BCNF (so also 3NF)
medical_reimbursement	reimbursement_id → hospital_name, treatment_type	{reimbursement_id}	BCNF (so also 3NF)
business_reimbursement	reimbursement_id → purpose, client_name	{reimbursement_id}	BCNF (so also 3NF)
expense_item	reimbursement_id, item_no → category, description, amount, expense_date, receipt_path	{reimbursement_id, item_no}	BCNF (so also 3NF)

Equally important are the dependencies that do **not** hold. Each was checked against the real-world meaning of the data, because assuming one by mistake would change the keys and the normal forms:

Dependency that does NOT hold	Reason (real-world fact)	Consequence for the design
name → employee_id	Two employees can share a name.	name is not a key; login uses email.
location → dept_id	Several departments are in Pune.	location is an ordinary attribute of department.
applicant_id → reimbursement_id	An applicant files many claims.	SUBMITS is one-to-many.
applicant_id, claim_date → reimbursement_id	Two claims may be filed on the same day.	No unique constraint on that pair.
reviewed_by → review_date	A manager reviews claims on many days.	review_date belongs to the review, i.e. the claim.

Dependency that does NOT hold	Reason (real-world fact)	Consequence for the design
travel_from, travel_to → distance_km	Distance is what was actually travelled on this trip (detours, return legs).	travel_reimbursement stays in BCNF; see decision E18.
designation → approval_limit	Limits are set per manager, not per job title.	manager stays in BCNF; see decision E17.
hospital_name → treatment_type	A hospital offers many treatments.	Both are attributes of the medical claim.

7. Normalization up to 3NF

7.1 Definitions

Normal form	Condition (Silberschatz §7.3, §7.8, Exercise 7.19)
1NF	Every attribute domain is atomic: no lists or sets inside a cell.
2NF	In 1NF, and no non-prime attribute depends on a proper subset of a candidate key (no partial dependency). The textbook covers 2NF only in Exercise 7.19, calling it "of historical significance"; we show it because the assignment asks for every step up to 3NF.
3NF	For every FD $\alpha \rightarrow \beta$ in F^+: it is trivial, or α is a superkey, or every attribute in $\beta - \alpha$ belongs to some candidate key. Equivalently for our purposes: no non-prime attribute depends transitively on a key.
BCNF	For every non-trivial FD $\alpha \rightarrow \beta$ in F^+, α is a superkey. BCNF implies 3NF.

7.2 Every relation of the final schema

Applying the definitions to every final relation:

- **1NF** holds everywhere: every column is a single atomic value. The one multivalued attribute (phone) has its own relation.
- **2NF** only matters for relations with a composite key: employee_phone and expense_item. employee_phone has no non-prime attributes. In expense_item no attribute depends on reimbursement_id alone (claim-level facts live in reimbursement) or on item_no alone (item numbers repeat across claims).
- **3NF and BCNF.** In every relation, the left side of every non-trivial FD in the table above is a candidate key (dept_name, email, (ifsc_code, bank_account_no) and payment_ref are keys too). Every relation is therefore in BCNF, which implies 3NF.

7.3 Walkthrough from an unnormalized claim form

To show that the schema is normalized (and not merely normal by luck), we start from the single flat relation a paper claim form or a spreadsheet would produce, and normalize it step by step.

```
claim_form (reimbursement_id, claim_type, claim_date, status,
            applicant_id, applicant_name, email, dept_id, dept_name, location,
            bank_account_no, ifsc_code, phones,
            reviewed_by, approval_limit, review_date, remarks,
            item_no, category, description, amount, expense_date)
```

Functional dependencies (F)

```
F1  reimbursement_id          -> claim_type, claim_date, status, applicant_id,
```

```
                                reviewed_by, review_date, remarks
F2  applicant_id             -> applicant_name, email, dept_id, bank_account_no, ifsc_code
F3  email                    -> applicant_id
F4  ifsc_code, bank_account_no -> applicant_id
F5  dept_id                  -> dept_name, location
F6  dept_name                -> dept_id
F7  reviewed_by              -> approval_limit
F8  reimbursement_id, item_no -> category, description, amount, expense_date
```

Candidate key. (reimbursement_id, item_no)$^+$ = all attributes: F8 gives the item columns, F1 the claim columns, then F2, F5 and F7 the rest. Neither attribute appears on the right of any FD, so every key must contain both: it is the only candidate key. Prime attributes: reimbursement_id, item_no.

Step 1: 1NF. `phones` holds a list (e.g. "+9198…, 020…"), which is not atomic. Move it to its own relation. (Repeating each row once per phone would instead create the multivalued dependency applicant_id ↠ phone and multiply every item row.)

```
employee_phone (applicant_id, phone)        key: (applicant_id, phone)
```

Step 2: 2NF. F1 is a partial dependency: claim_type, claim_date, status, applicant_id, … depend on reimbursement_id alone, a proper part of the key (and the applicant, department and reviewer columns follow from them). Split the item-level and claim-level facts:

```
expense_item (reimbursement_id, item_no, category, description, amount, expense_date)
claim        (reimbursement_id, claim_type, claim_date, status, applicant_id, applicant_name,
              email, dept_id, dept_name, location, bank_account_no, ifsc_code,
              reviewed_by, approval_limit, review_date, remarks)
```

Lossless: expense_item ∩ claim = {reimbursement_id}, which is a key of claim (§7.2.3 test). expense_item is now in BCNF: its only non-trivial FD is F8, whose left side is its key.

Step 3: 3NF. claim (key reimbursement_id) still has transitive dependencies: reimbursement_id → applicant_id → applicant_name, email, …; applicant_id → dept_id → dept_name, location; and reimbursement_id → reviewed_by → approval_limit. Each makes a non-prime attribute depend on a non-key attribute. Decompose on F2, F5 and F7:

```
reimbursement     (reimbursement_id, claim_type, claim_date, status, applicant_id,
                   reviewed_by, review_date, remarks)
applicant_details (applicant_id, applicant_name, email, dept_id, bank_account_no, ifsc_code)
department        (dept_id, dept_name, location)
manager           (reviewed_by -> employee_id, approval_limit)
```

Lossless: each split shares an attribute that is a key of one side (applicant_id of applicant_details, dept_id of department, reviewed_by of manager).

Dependency preservation. Every FD lies inside one final relation: F1 in reimbursement; F2, F3, F4 in applicant_details; F5, F6 in department; F7 in manager; F8 in expense_item. So each can be checked without a join (§7.4.4).

Cross-check with the 3NF synthesis algorithm (§7.5.2). F is already a canonical cover (no extraneous attributes, one FD per left side). One schema per FD gives the relations above (F3, F4 and F6 fall inside F2's and F5's schemas and are dropped), and the candidate key (reimbursement_id, item_no) is contained in expense_item, so no extra key schema is needed. The algorithm produces the same result.

> **Result**
> The final relations are exactly the schema we derived from the ER diagram, with one difference: normalization alone keeps applicant_details as one relation. Our design splits it into **employee** and **applicant** because of the partial specialization: employees who are not applicants (an intern, a finance clerk) would otherwise need NULL bank details. Both parts are in BCNF.

> The payment columns (paid_by, payment_date, payment_ref) and the claim-type details were left out above to keep it short. They behave the same way: payment columns depend on reimbursement_id only, and the type-specific details move to the subclass relations to avoid NULLs, not because of any normal form.

8. Implementation

8.1 Architecture

Layer	Technology	Responsibility
Front end	React 19, React Router, Tailwind CSS (Vite)	Forms and screens for the three roles; shows database errors as messages.
Back end	Node.js, Express 5, mysql2	Authentication (bcrypt, JWT), role checks, transactions, file uploads; every query parameterised.
Database	MySQL 8.0 (InnoDB)	11 tables, 13 foreign keys, 4 unique keys, 18 CHECK constraints, 17 triggers, 2 views, 1 function, 2 procedures.

8.2 Where each rule is enforced

Mechanism	Rules (Section 3.3)	Examples
Primary and unique keys	DR9, one decision per claim, item numbers per claim	uq_employee_email, uq_applicant_bank, uq_reimbursement_payment_ref
Foreign keys (RESTRICT / CASCADE)	DR1, DR11, weak entity existence	fk_reimbursement_reviewer → manager; fk_item_reimbursement ON DELETE CASCADE
CHECK constraints	DR2, DR5, DR6 (amount > 0), DR8 (order of dates), domains	chk_reimbursement_lifecycle, chk_reimbursement_no_self_review
Triggers	DR3, DR4, DR6 (one type, at least one item), DR7, DR8 (not in future), DR10	trg_reimbursement_bu, trg_expense_item_bd, trg_employee_bu
Transactions (server)	Claim, details and items saved atomically	POST /api/claims

Rules need a trigger when a CHECK cannot express them: they read other rows or tables, compare old and new values, or need today's date. MySQL, like most systems, does not support SQL assertions (textbook §4.4.8). db/06_constraint_tests.sql runs 50 tests that try to break each rule and confirms the database rejects every one.

8.3 Screens

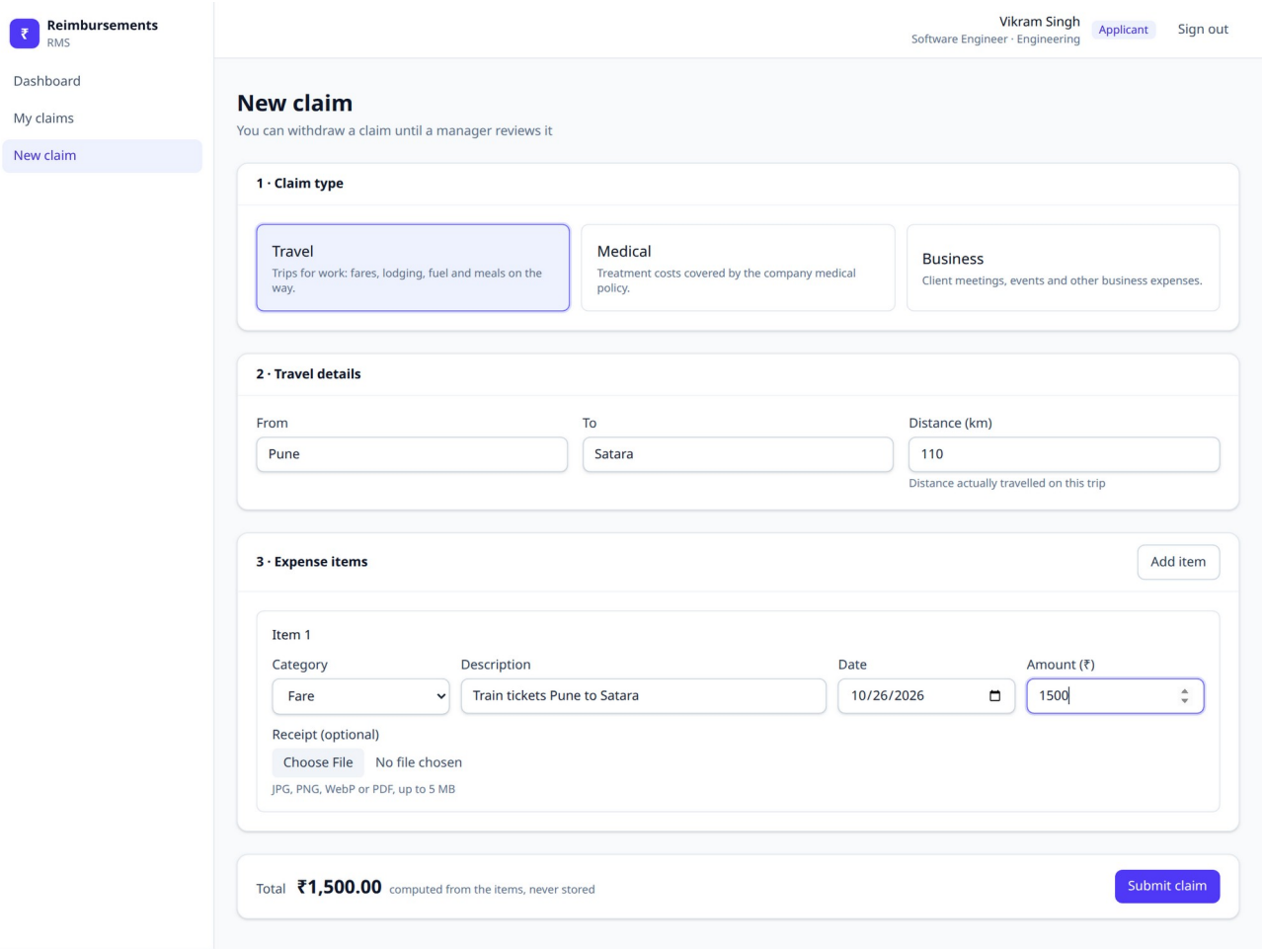


Figure 3. New claim: one claim type with its details, then any number of expense items.

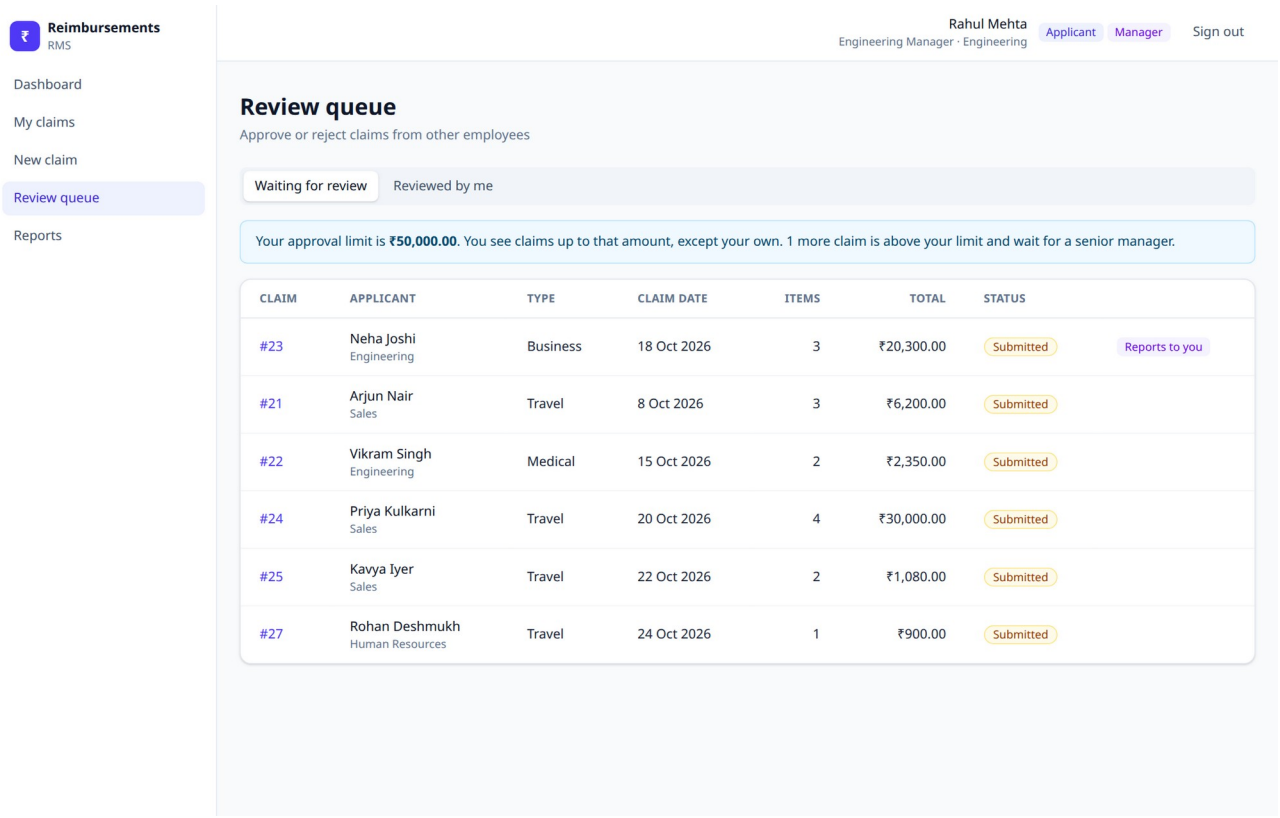


Figure 4. Review queue: only claims the manager is allowed to decide.

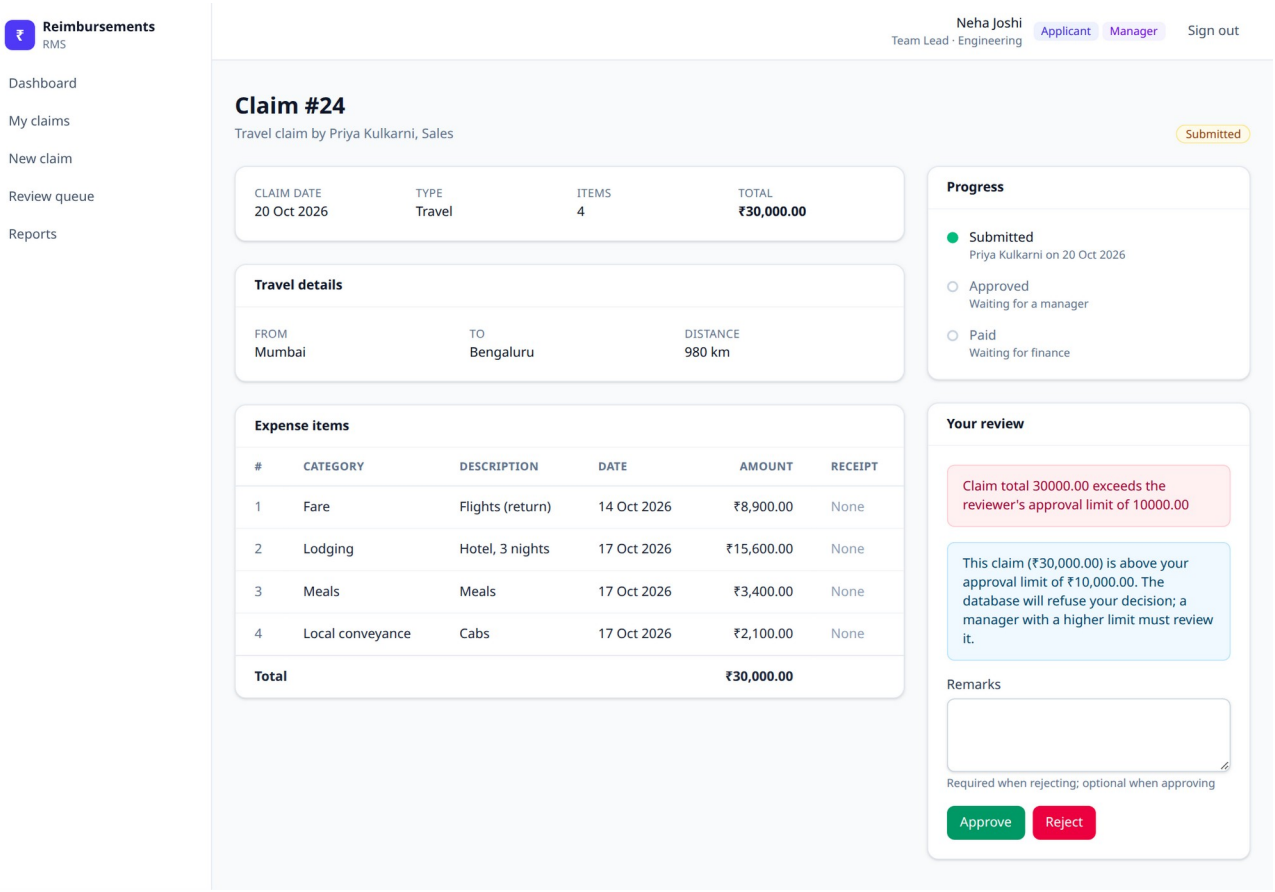


Figure 5. A manager with a ₹10,000 limit tries to approve a ₹30,000 claim; the trigger's message is shown.

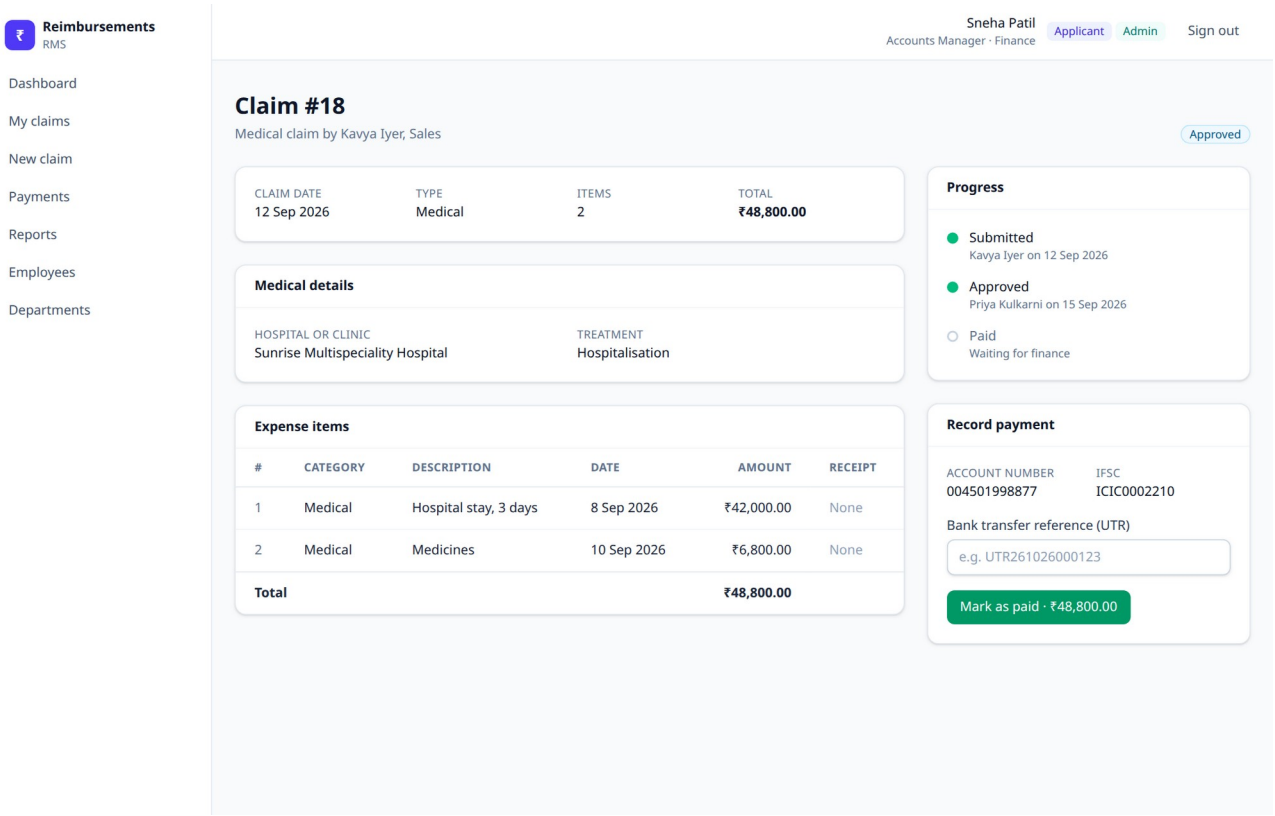


Figure 6. Recording a payment against an approved claim.

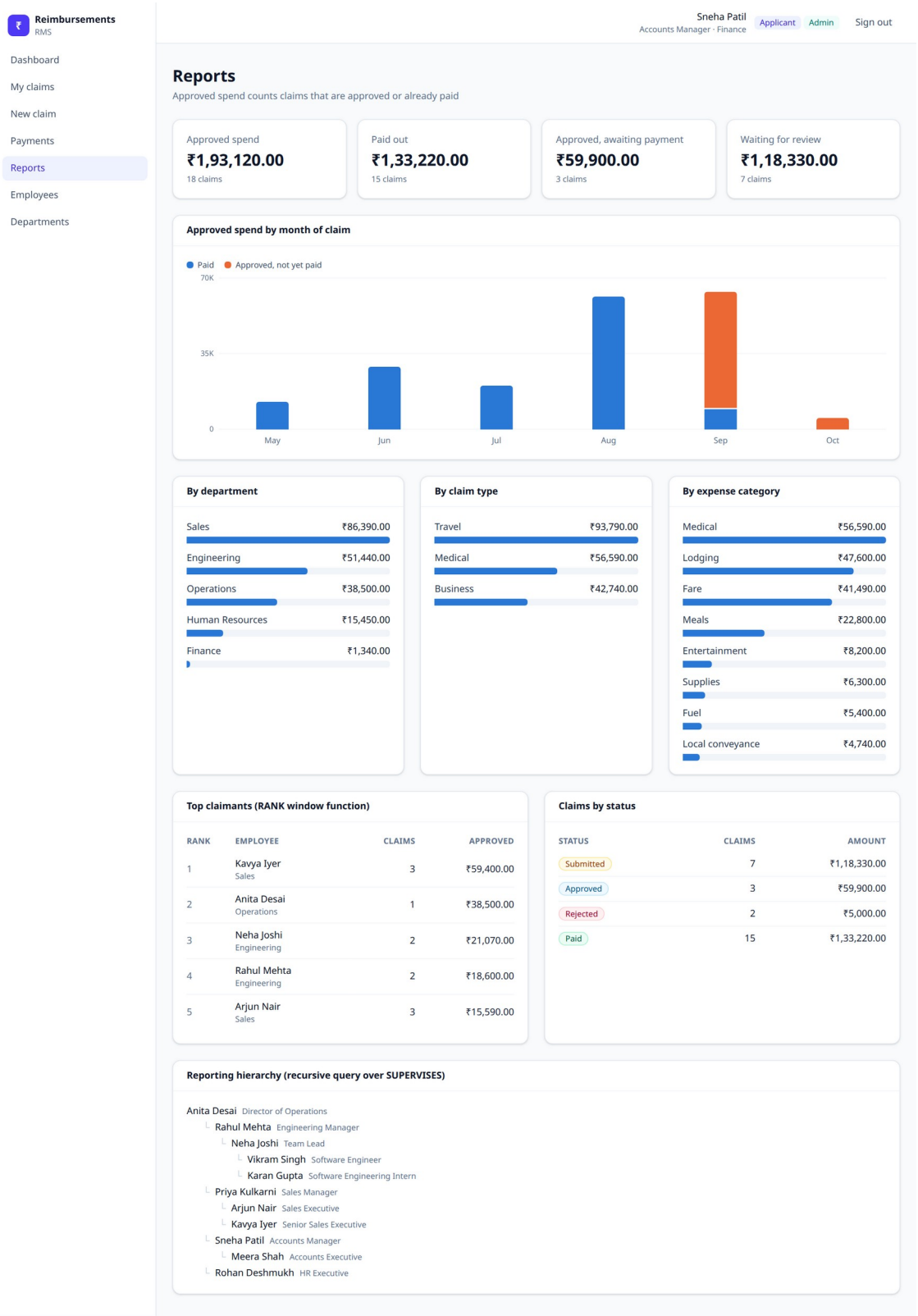


Figure 7. Reports: grouping, a window function (RANK) and a recursive query over SUPERVISES.

8.4 Running the project

```
docker compose up -d                                    # MySQL 8.0 on port 3307 (or use a local
MySQL)
./db/setup.sh                                           # schema, views, triggers, app user, sample
data
mysql -h127.0.0.1 -P3307 -uroot -p --protocol=TCP < db/06_constraint_tests.sql   # 50 tests
cd server && cp .env.example .env && npm install && npm start                    # API on :4000
cd client && npm install && npm run dev                                          # UI on :5173
```

Sample accounts all use the password password123 (for example vikram.singh@rms.example as an applicant, rahul.mehta@rms.example as a manager, sneha.patil@rms.example as an admin). The repository README lists all of them.

9. Conclusion

The database was designed top-down from an E-R model in the textbook's notation, reduced to 11 relation schemas with the standard rules, and checked against its functional dependencies: every relation is in BCNF, and therefore in 3NF. A walkthrough from a flat claim form reaches the same schema through 1NF, 2NF and 3NF, with lossless and dependency-preserving decompositions. Business rules are enforced in the database by keys, CHECK constraints and triggers, and a web application for all three roles runs on top of it.

> **Further detail**
> The reasoning behind every design decision, the alternatives we rejected and the trade-offs we accepted are in the companion document, RMS_Design_Decisions.pdf.